Security Audit Report — JMDN (JMDT Decentralised Node)

Date: 2026-07-15

Auditor: Claude (AI Security Review)

Scope: Static source review of the JMDN Go L2 blockchain node — consensus (AVC/BLS/BFT), cryptography & signatures, transaction processing (Sequencer/Mempool/Security), P2P networking (libp2p/seednode/transfer), RPC/WS/gRPC APIs (gETH/CLI/explorer), dependencies (`go.mod`), and infrastructure (Dockerfile, docker-compose, CI, config).

Method: Static analysis only. No runtime testing, fuzzing, or live exploitation. ~108k LOC across 465 Go files.

Executive Summary

JMDN has a **critically broken consensus authentication path** and an **unauthenticated remote-file-write primitive**, either of which is independently fatal. The live BLS consensus signs a constant string (`"vote:1"`) bound to no block, no round, and no committee — any captured or self-generated signature approves any block, forever (consensus forgery + universal replay). Separately, the P2P file-transfer handler writes attacker-controlled filenames to disk with no path confinement, giving any peer arbitrary file write (RCE/persistence). A committed BLS private key sits in the repo.

Beyond those, the network surface is authenticated almost entirely by bind-address/firewall: nearly every service defaults to `AuthType: none` and `TLS: false`, rate limiting ships disabled, and the admin CLI gRPC (full node control) has no auth with reflection enabled. Multiple signature-verification routines exist but are **never called** (peer records, heartbeats, the entire Ed25519 BFT engine).

Overall risk posture: **Critical — not production-safe.** The consensus layer, the primary security guarantee of a blockchain, is forgeable today. Prioritize C1–C3 and H1 before any mainnet exposure.

Note on a prior in-repo audit: `audits/2026-03-terasoft-certin-vapt/` contains a CERT-IN VAPT certificate. The findings below are not reflected as fixed in the current tree; treat this review as superseding it.

Findings Summary

Severity	Count
■ Critical	3
■ High	9
■ Medium	12
■ Low	6
■■ Info	3

Findings

■ *CRITICAL*

[CRITICAL] C1 — BLS consensus votes bind to nothing → consensus forgery & universal replay

Location: `AVC/BuddyNodes/MessagePassing/BLS_Signer/Signer.go:47`; `AVC/BuddyNodes/MessagePassing/BLS_Verifier/Verifier.go:13-38`; `Sequencer/Consensus.go:1988-2043`

Pillar: Code Vulnerability / Consensus

Description:

The canonical message signed for a consensus vote is a constant string that encodes only the vote value — no block hash, block number, round, chain ID, or signer identity. The verifier trusts the public key supplied *in the same response*, never checks it against an authorized committee, does not deduplicate by signer, and reaches "consensus" on a simple majority of whoever replied (`validTotal/2 + 1`) rather than 2/3 of a fixed committee.

Evidence:

```
// BLS_Signer/Signer.go:47
msg := []byte("vote:" + strconv.Itoa(int(vote)))   // "vote:1" or "vote:-1" — binds nothing

// Sequencer/Consensus.go (VerifyConsensusWithBLS)
if err := BLS_Verifier.Verify(r, vote); err != nil { continue }  // verifies sig over "vote:1" for r.PubKey (attacker
validTotal++; if vote == 1 { validYes++ }
needed := (validTotal / 2) + 1
if validYes >= needed { return true }               // simple majority of responders, no committee allowlist
```

Exploit: (1) Replay a single "yes" signature captured from any validator to approve any malicious block, forever. (2) Generate N BLS keypairs, sign `"vote:1"` with each, return them as vote results — all verify, `validYes = validTotal = N`, consensus "reached" for an arbitrary block. No authorized-key set stops this.

Remediation: Sign `H(chainID ■ blockNumber ■ blockHash ■ round ■ vote)`. In the verifier: require each pubkey to be in the registered committee for that block and to match the claimed PeerID; deduplicate by validator identity; require `validYes*3 >= committeeSize*2` over the fixed committee. Route live consensus through the existing `MultiSigManager` (`AVC/BLS/bls-sign/bls-sgin.go:230-260`), which already verifies-then-records and dedups by signer, and the correct 2/3 helpers in `AVC/BFT/bft/math.go` (`HasQuorum`).

[CRITICAL] C2 — Committed BLS private keys (consensus signing keys) in the repo

Location: `AVC/BLS/Router/config/bls.json:2`; `AVC/BuddyNodes/MessagePassing/BLS_Signer/config/bls.json:2`; `AVC/BuddyNodes/MessagePassing/BLS_Verifier/config/bls.json:2` (all confirmed `git ls-files`-tracked)

Pillar: Secrets & Credentials

Description:

Three 32-byte base64 BLS private keys are committed to the repository. `AVC/BLS/bls-sign/bls-sgin.go:92-107` reads `bls_priv` from `config.BLSFile` (`config/constants.go:24` = `./config/bls.json`), base64-decodes it, and uses it directly as the signer key.

Evidence:

```
// AVC/BLS/Router/config/bls.json  (tracked in git)
{ "bls_priv": "KwV43FiVHEYpno9N2h8CglstAS3z3FkKB3tojIrMJZs=", "bls_pub": "K6OG7iR9..." }
```

Remediation: Treat all three keys as permanently compromised — rotate/regenerate. Remove the files and purge from git history (`git filter-repo`). Add `**/config/bls.json` to `.gitignore` (only the runtime path is ignored today). The code already supports auto-generate + persist (`bls-sgin.go:110-140`) — generate at deploy time.

[CRITICAL] C3 — Arbitrary file write via unauthenticated P2P file transfer (path traversal)

Location: `transfer/file.go:270-303`; handler registered at `node/node.go:201-202` and `main.go:1246-1248` (`config.FileProtocol` = `/custom/file/1.0.0`, P2P port 15000)

Pillar: Code Vulnerability

Description:

The file-stream handler takes the destination filename from the wire (up to 1024 bytes, attacker-controlled) and, because all registered callers pass `outputPath == ""`, uses it **raw** as the output path. `filepath.Base` is applied only on the *other* branch. No base-directory confinement, no `../` rejection, absolute paths accepted. The declared file size is also unbounded (see H9).

Evidence:

```
filename = string(header[16 : 16+filenameLen])   // attacker-controlled
if outputPath == "" {
    if filename != "" { outputPath = filename }   // raw — no Base(), no confinement
    ...
}
os.MkdirAll(filepath.Dir(outputPath), 0750)
file, err := os.Create(outputPath)                // writes anywhere the node user can
```

Exploit: Any peer opens a libp2p stream with `filename = "../../../../root/.ssh/authorized_keys"` or `/etc/cron.d/x` → arbitrary file write as the node user → RCE/persistence. Unauthenticated.

Remediation: Always `filepath.Base(filename)`; join under a fixed dedicated download directory; reject `..`, absolute paths, and empty names; re-validate the final path with `filepath.Clean` + prefix check. Authenticate the file protocol.

■ *HIGH*

[HIGH] H1 — Public RPC admits unsigned transactions via JSON path + "internal deployment" bypass

Location: `gETH/Facade/Service/Service.go:395-418` (`SendRawTx`); `Block/Server.go:205-232` (`SubmitRawTransaction`); `config/ZKBlock.go:10-33`

Pillar: Code Vulnerability

Description:

`SendRawTx` first attempts `json.Unmarshal` of client bytes straight into `config.Transaction`, whose JSON tags expose `from`, `to`, `value`, `nonce`, `hash`, `v/r/s` — so a client controls every field, including `From`, with no signature. `SubmitRawTransaction` then bypasses all security checks for any tx with `To == nil && V == nil`, trusting it as an "internal deployment."

Evidence:

```
// Block/Server.go:205
isInternalDeployment := tx.To == nil && tx.V == nil
if isInternalDeployment {
    // "bypassing signature validation" — trusted as internal
} else {
```

```
    status, err := Security.AllChecks(tx)   // sig/hash/balance/nonce only on this branch
}
```

Exploit: POST a hex-encoded JSON tx with `to:null`, `v:null`, arbitrary `from`/`data`/`hash`. It skips `Security.AllChecks` entirely and enters the deploy pipeline attributed to a spoofed deployer. (Value transfers with `To != nil` *are* caught — `Security.CheckSignature` correctly recovers/matches the sender.)

Remediation: Remove the client-controlled JSON ingest path from public RPC (accept RLP only) or run `Security.AllChecks` on it. Eliminate the `V == nil` field-absence trust heuristic; authenticate internal deployments by loopback/process origin or an internal signing key.

[HIGH] H2 — Signed peer records / heartbeats / aliases / neighbors are never verified

Location: `seednode/signature.go` (validators defined); `seednode/seednode.go:462,654,670,772,794,887,935` (signers called)

Pillar: Code Vulnerability / P2P

Description:

`SignPeerRecord`/`SignHeartbeat`/`SignAlias`/`SignNeighbor` are invoked, but the matching `Validate*Signature` functions are **never called anywhere** (dead code). Peer records, heartbeat status transitions, aliases, and neighbor topology edges are accepted without verification.

Exploit: Forge/replace another node's advertised multiaddrs, flip peer status, or inject fake neighbor topology → eclipse/routing manipulation and identity spoofing across the seednode layer.

Remediation: Call the matching `Validate*Signature` on every inbound record/heartbeat/alias/neighbor before trusting it; reject on failure. Fix the malleable R/S encoding first (M11).

[HIGH] H3 — Ed25519 BFT engine never signs; gossip receivers accept unauthenticated votes

Location: `AVC/BFT/bft/engine.go:40-53,363-386`; `AVC/BFT/bft/bft.go:24-70`; `AVC/BFT/bft/bft_pubsub_adapter.go:119,155-210`; `Sequencer/Triggers/Triggers.go:539-547`

Pillar: Code Vulnerability / Consensus

Description:

`RunConsensus` receives a `Signer` but never stores or uses it; `PrepareMessage`/`CommitMessage` are built with an empty `Signature`; both call sites pass `nil`. Verification is gated on `config.RequireSignatures` (default true), so the engine is either non-functional or must be run with signatures disabled. The gossip receivers `HandlePrepareVote`/`HandleCommitVote` enqueue votes from any peer with zero authentication. `round` is hardcoded to `1` and `lastSeqSeen` resets per run, defeating replay protection.

Exploit: If `RequireSignatures` is set false (or the missing-signature guard removed to make it "work"), any peer publishes forged PREPARE/COMMIT votes and drives the outcome; buddy-ID membership is the only gate.

Remediation: Actually invoke the signer before broadcast; store it on the engine; make `RequireSignatures` non-overridable in production; bind digests to a persistent monotonic (height, round).

[HIGH] H4 — Facade debug server has no auth (full chain-projection DB reads)

Location: `gETH/Facade/rpc/debug_server.go:17-22`; routes in `gETH/Facade/rpc/thebe_read_routes.go:24-38`

Pillar: Code Vulnerability / API

Description:

The debug router is `gin.New()` with only Logger/Recovery/CORS — no gatekeeper middleware (unlike the main RPC server). It exposes `/debug/dualdb/report` plus all Thebe read routes: full block/account/tx/zkproof/snapshot projection reads and account nonces. Default bind is `127.0.0.1`, but there is zero defense-in-depth: a single `binds.thebedebug: 0.0.0.0` (or container port publish) world-exposes the entire chain DB with wildcard CORS and no auth.

Remediation: Apply the `gatekeeper` middleware to the debug router; require token/mTLS; never rely on bind address alone.

[HIGH] H5 — WebSocket RPC: no auth, no TLS, origin wide-open, unbounded subscriptions

Location: `gETH/Facade/rpc/ws_server.go:31,40-45,82,101-160`

Pillar: Code Vulnerability / API

Description:

`CheckOrigin` always returns true (cross-site WebSocket hijacking). The server uses a raw `http.ServeMux` + `ListenAndServe`, bypassing `gatekeeper.ConfigureNetHTTPServer` (no rate limit, no TLS), bound `0.0.0.0:8546`. Each `eth_subscribe` spawns a goroutine with no per-connection cap (memory/goroutine exhaustion). Subscription IDs are derived from a microsecond timestamp (`sid := "0x" + time.Now()...`) — concurrent subs collide and overwrite each other's stop functions. No WS read-size limit.

Remediation: Wrap with gatekeeper middleware + TLS; restrict `CheckOrigin` to an allowlist; cap subscriptions per connection; use crypto-random sub IDs; set `conn.SetReadLimit`.

[HIGH] H6 — JSON-RPC accepts unbounded request bodies (memory DoS / slowloris)

Location: `gETH/Facade/rpc/http_server.go:75-79,132`

Pillar: Code Vulnerability / API

Description:

`c.GetRawData()` reads the entire body into memory. `ReadHeaderTimeout` is set (10s) but there is no `ReadTimeout` and no `http.MaxBytesReader`. Batch count is capped at 100 but each element and the raw body are size-unbounded.

Exploit: A single multi-GB POST or slow body → OOM / connection exhaustion on the public RPC port.

Remediation: Wrap body with `http.MaxBytesReader`; set `ReadTimeout`/`WriteTimeout`; cap per-request size.

[HIGH] H7 — Admin CLI gRPC: full node control, no auth, reflection enabled

Location: `config/settings/security.go:110-114` (`ServiceCLI` = TLS:false, Auth:None); `CLI/GRPC_Server.go:374,389`

Pillar: Code Vulnerability / API

Description:

The admin gRPC service defaults to no TLS and no auth (the code even logs "TLS is disabled for Admin service - THIS IS INSECURE") and registers gRPC reflection. Surface includes `AddPeer`/`RemovePeer`, `FastSync`/`FastSyncV2`, `CatchUpSync`, `RebuildTxIndex(Range)`, `SendFile` (arbitrary local path → exfiltration), `SendMessage`/`Broadcast`. Sole protection is `binds.cli: 127.0.0.1`; because policy is `None`, gatekeeper provides zero protection if the port is ever reachable (misbind, container publish, SSRF pivot) → full node takeover.

Remediation: Set `ServiceCLI` to mTLS (or token) even on localhost; disable reflection in production.

[HIGH] H8 — Hardcoded well-known test mnemonic derives node-selection identity

Location: `AVC/NodeSelection/Router/Router.go:21-22,31`; `AVC/NodeSelection/pkg/selection/keys.go:14-30`

Pillar: Secrets & Credentials / Consensus

Description:

```
const mnemonic = "abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about"
const networkSalt = "test-salt"
```

This is the canonical BIP-39 zero-entropy test mnemonic. It is deterministically derived into an ed25519 keypair used for buddy-node/VRF selection, so every node ships the same, publicly-known identity, with a fixed salt.

Exploit: Reproduce the key and salt to impersonate nodes or bias/predict buddy-node (VRF) selection. Compounds with M4 (predictable VRF).

Remediation: Load the mnemonic/private key from secure per-node config/env (`keys.go:43` `LoadKeysFromConfig` already supports `PRIVATE_KEY`/`MNEMONIC`); make `networkSalt` a network-wide configured secret.

[HIGH] H9 — Unbounded declared file size over P2P → disk-fill DoS

Location: `transfer/file.go:260,322-393`

Pillar: Code Vulnerability / P2P

Description:

`fileSize := binary.LittleEndian.Uint64(header[:8])` is used to drive the write loop with no upper bound. A malicious peer streams data until the disk fills (evicting the DB). Unauthenticated, same handler as C3.

Remediation: Enforce a max file size and per-peer quota; abort when exceeded.

[HIGH] H10 — Rate limiting disabled and forwarded-header spoofing enabled in shipped config

Location: `jmdn_default.yaml:135-139,159-224`; `jmdn.yaml:135-139` (working tree)

Pillar: Infrastructure / Config

Description:

Default config sets `global_rate_limit: 0`, `global_burst: 0`, and per-service `rate_limit: 0` (unlimited), on a public `eth_rpc` that also has `auth_type: none`. Additionally `trust_forwarded_headers: true` with empty `trusted_proxies: []` — `X-Forwarded-For` is trusted from any source, letting clients spoof source IP and defeat any per-IP control. Only `jmdn_exchange.yaml` sets non-zero limits.

Remediation: Set non-zero global + per-public-service limits in the default config (mirror the exchange profile). Populate `trusted_proxies` with real LB CIDRs, or set `trust_forwarded_headers: false` when not behind a trusted proxy.

[HIGH] H11 — Postgres default password + host-wide exposure + TLS disabled

Location: `docker-compose.yml:64,71-72,103,111,268`; `jmdn_default.yaml:54`

Pillar: Infrastructure / Config

Description:

`POSTGRES_PASSWORD: "${POSTGRES_PASSWORD:-jmdndefault}"` and Redis `--requirepass ${REDIS_PASSWORD:-jmdnredissync}` ship with known default fallbacks — if the operator omits `.env`, the stack comes up with public credentials. Postgres is published as `"5430:5432"` (binds `0.0.0.0` on the host), and the DSN uses `sslmode=disable` (cleartext). Default password + host-wide exposure = remote DB compromise.

Remediation: Remove `:-default` fallbacks so compose fails closed (`${POSTGRES_PASSWORD:?set in .env}`); bind Postgres to `127.0.0.1:5430:5432` (jmdn reaches it over the compose network); use `sslmode=require`/`verify-full` off-loopback.

■ *MEDIUM*

[MEDIUM] M1 — Weak dev secrets in on-disk config `jmdn.yaml`

Location: `jmdn.yaml:138-139,145` (working tree — **gitignored, not committed**)

Pillar: Secrets & Credentials / Config

Description: The active on-disk config contains `explorer_api_key: "testkey123"`, `jwt_secret: "localdevsecret"`, and `explorer_api tls: false` (with a comment noting the requirement is `true`). This file is `.gitignore`d and confirmed untracked, so it is **not** a committed-secret exposure — but it is the config the node/container loads, and `localdevsecret` is trivially guessable (JWT forgery) if it ever reaches a deployed node. The tracked templates correctly ship empty values.

Remediation: Rotate both secrets; inject via env (`JMDN_SECURITY_JWT_SECRET`, `JMDN_SECURITY_EXPLORER_API_KEY`); set `tls: true`. Ensure this file is never force-added.

[MEDIUM] M2 — Nil-pointer panic (node crash) on JSON tx with missing `value`

Location: `Block/Server.go:236`

Pillar: Code Vulnerability

Description: `if tx.Value.Cmp(big.NewInt(0)) == 0 || ...` dereferences `tx.Value` (`*big.Int`). Via the JSON ingest path (H1) a client can omit `value`, leaving it `nil` → panic. Reachable from the public `eth_sendRawTransaction` RPC.

Remediation: Nil-check `tx.Value` (and other `*big.Int` fields) before use; reject malformed txs.

[MEDIUM] M3 — Future-nonce acceptance

Location: `Security/Security.go:511-517`

Pillar: Code Vulnerability

Description: Only `tx.Nonce < expectedNonce` is rejected; a `TODO(nonce-gap)` acknowledges `tx.Nonce > expectedNonce` is accepted. Committing such a tx jumps the account nonce and orphans intermediate nonces, enabling gap-based mempool manipulation.

Remediation: Enforce strict `tx.Nonce == expectedNonce`, or an explicit bounded queue.

[MEDIUM] M4 — Weak/deterministic randomness in VRF buddy selection (predictable, static)

Location: `AVC/NodeSelection/pkg/selection/vrf.go:5,52,99,105-115,168`

Pillar: Cryptographic Weakness

Description: Uses `math/rand` for Fisher-Yates shuffles seeded with a constant `rand.NewSource(0)`. More importantly, `buildRoundMessage` is `nodeID + ":" + networkSalt` with no round/epoch/height, so the VRF output (and selected buddy set) is static and fully predictable per node for the life of the salt.

Exploit: Precompute which peers any node will select and position Sybil nodes to dominate a target's buddy set. Compounds with H8.

Remediation: Include a per-round/epoch beacon in the VRF input; derive shuffles from a VRF-derived seed only once the input rotates; use `crypto/rand` where unpredictability matters.

[MEDIUM] M5 — GossipSub: no topic validators, no max message size, flood-publish on

Location: `config/PubSubMessages/GossipSub_Helper.go:41-44`

Pillar: Code Vulnerability / P2P

Description: `pubsub.NewGossipSub(... WithFloodPublish(true), WithPeerExchange(true))` with no `RegisterTopicValidator`, no `WithMaxMessageSize`, and no signature policy anywhere in the repo. Messages are accepted/relayed with libp2p defaults and no application-layer validation before propagation; flood-publish amplifies malicious/oversized messages across the mesh.

Remediation: Register per-topic validators that verify size/semantics/signature before propagation; set an explicit max message size; consider disabling flood-publish on large networks.

*[MEDIUM] M6 — Wildcard CORS (Explorer also sets Allow-Credentials with *)*

Location: `gETH/Facade/rpc/http_server.go:194`; `explorer/api.go:369,372`; `Block/Server.go:375`; debug server (inherits `withCORS`)

Pillar: Config

Description: `Access-Control-Allow-Origin: *` on multiple surfaces. Explorer additionally sets `Access-Control-Allow-Credentials: true` alongside `*`, broadening credentialed cross-origin read / CSRF surface for any origin against JWT-protected data.

Remediation: Replace `*` with an explicit allowlist on Explorer + debug; drop `Allow-Credentials` unless echoing a specific allowed origin.

[MEDIUM] M7 — Unauthenticated state-changing / expensive endpoints on public RPC

Location: `gETH/Facade/rpc/sync_handlers.go:19,33-43` (`/sync/reconcile`); `gETH/Facade/rpc/handlers.go:571-586` (`debug_traceTransaction`); `handlers.go:41-64` →

`SmartContract/pkg/compiler/compiler.go:83` (`solc_compile`)

Pillar: Code Vulnerability / API

Description: `/sync/reconcile` runs a synchronous Merkle build + seednode report with no auth (CPU DoS on repeat). `debug_traceTransaction` exposes expensive opcode tracing with no auth. `solc_compile` spawns `exec.Command("solc", "--standard-json", tmpfile)` on unauthenticated, size-unbounded source (not argument injection — fixed args — but full parser + unbounded CPU/mem exposed to the internet).

Remediation: Require auth + rate limits on these; make `/sync/reconcile` admin-only; gate/disable `solc_compile` and `debug_traceTransaction` on public nodes; cap source size; sandbox `solc` with CPU/mem/time limits.

[MEDIUM] M8 — Unauthenticated internal gRPC services (DID registration → Sybil/spam)

Location: `config/settings/security.go:116-144` (DID, gETH gRPC, Mempool, BlockIngestGRPC, BFT all Auth:None, TLS:false); `PORTS.md` §5

Pillar: Code Vulnerability / API

Description: All internal/validator gRPC services default to no auth and no TLS; only bind/firewall protects them. Per PORTS.md, `RegisterDID` is unauthenticated and public on resolver nodes — anyone reaching DID `:15052` can register arbitrary DIDs (Sybil/spam).

Remediation: mTLS for validator/internal gRPC; auth + rate limit + registration cost on `RegisterDID`.

[MEDIUM] M9 — Bootstrap snapshot integrity uses MD5 over a public bucket, unsigned

Location: `Scripts/bootstrap_sync.sh:41,109-121`; `docker-compose.yml:159-164`

Pillar: Infrastructure

Description: Chain snapshot parts and a `checksums.md5` are fetched from a public GCS location over HTTPS and verified with `md5sum -c`. MD5 has broken collision resistance and the checksum file is fetched from the same location as the data with no signature — anyone who can write the bucket (or MITM a misconfigured mirror) can substitute snapshot + matching MD5. Chain-state provenance is trust-critical.

Remediation: Sign the manifest (minisign/GPG/cosign), verify with a pubkey baked into the image; switch checksums to SHA-256.

[MEDIUM] M10 — gosec/Sonar security scanning disabled; Scripts/ excluded

Location: `.golangci.yml:32-34`; `.github/workflows/sonarqube.yml:15` (`if: false`); `sonar-project.properties:11`

Pillar: Infrastructure / Process

Description: `gosec` is commented out ("192 violations in current backlog"), along with `errcheck` and `staticcheck`; SonarQube is hard-disabled (`if: false`); `sonar.exclusions` blanks `Scripts/**` (the privileged entrypoint + cert/bootstrap shell — highest-risk scripts). No static security scanning runs in CI, so exactly the class of issues in this report goes uncaught.

Remediation: Run `gosec` in diff mode now (`--new-from-rev`), burn down the backlog, then enforce; re-enable Sonar; keep Scripts/ in scope.

[MEDIUM] M11 — Malleable custom ECDSA signature encoding (seednode)

Location: `seednode/signature.go:16-21,170-178,218-226,265-273,358-366`

Pillar: Cryptographic Weakness

Description: Signatures store R/S as `big.Int` hex; `big.Int.Bytes()` strips leading zeros, and reconstruction pads the concatenation to 64 bytes, misaligning whenever R has leading zeros and S does not. `calculateVFromSignature` fabricates a "V" from `(r+s)&1` with no recovery meaning. Yields malleable/ambiguous encodings; latent today because these records aren't verified (H2) but must be fixed alongside it.

Remediation: Use fixed-width 32-byte big-endian R/S (or libp2p native signature bytes); verify with the same canonical encoding.

[MEDIUM] M12 — GitHub Actions pinned to mutable tags, not commit SHAs

Location: `.github/workflows/docker.yml`, `ci.yml`, `sonarqube.yml`

Pillar: Infrastructure / Supply Chain

Description: Third-party actions are pinned to mutable tags (`actions/checkout@v4`, `docker/build-push-action@v5`, SonarSource `@v6/@v1`, etc.). A hijacked tag executes arbitrary code in the release workflow, which holds `packages: write` + `GITHUB_TOKEN` and builds/pushes the production image.

Remediation: Pin third-party actions to full commit SHAs (`uses: actions/checkout@<sha> # v4.x`).

■ *LOW*

[LOW] L1 — `ethtypes.Sender` error discarded at tx conversion

Location: `gETH/Facade/Service/Service.go:444` — `from, _ := ethtypes.Sender(...)`. On an unverifiable signature this yields a zero/garbage `from` with the error swallowed. Downstream `Security.AllChecks` re-verifies signed txs (defense-in-depth), but combined with H1 the `From` at this layer is never guaranteed to match the signature. **Fix:** check and propagate the error; drop on failure.

[LOW] L2 — Verbose internal errors returned to clients

Location: `gETH/Facade/rpc/thebe_read_routes.go:58,82,102,129,148,171,190,213,236,260,279`; `ws_server.go:114,134,147`. Raw `err.Error()` leaks DB driver / internal detail. **Fix:** return generic messages; log details server-side.

[LOW] L3 — Notably old / unmaintained dependencies

Location: `go.mod`. `github.com/yahoo/coname v0.0.0-20170609175141` (2017, crypto-adjacent, abandoned — highest concern); `github.com/gogo/protobuf v1.3.2` (deprecated, past CVEs); `github.com/shirou/gopsutil v3.21.11+incompatible` (2021); `github.com/mitchellh/mapstructure v1.5.0` (archived). Security-critical libs are current (x/crypto v0.52.0, grpc v1.79.3, quic-go v0.59.0, go-libp2p v0.47.0, geth v1.17.0, jwt/v5 v5.3.1). **Fix:** drop `yahoo/coname` and `gogo/protobuf`; run `govulncheck` to confirm reachable CVEs.

[LOW] L4 — Internal gRPC clients hardcode `insecure.NewCredentials()`

Location: `Block/gRPCclient.go:40`; `Block/Singleton_RoutingClient.go:31`; `SmartContract/cmd/main.go:111,121`; `SmartContract/server_integration.go:48,65`; `SmartContract/pkg/client/client.go:22`; `DB_OPs/contractDB/contractdb.go:471`; `seednode/seednode.go:121`. Plaintext gRPC contradicts the yaml mTLS policy for these paths. **Fix:** route through `pkg/gatekeeper` TLS; keep insecure only for verified loopback.

[LOW] L5 — Container entrypoint runs as root before dropping to `jmdn`

Location: `Dockerfile:149`; `Scripts/docker-entrypoint.sh` (gosu → uid 3322). Intentional (bootstrap chown), mitigated by `no-new-privileges:true`. **Fix:** pre-chown at build; add `USER jmdn` where bootstrap chown isn't needed.

[LOW] L6 — Base image pinned by tag, not digest; compose default `:latest`

Location: `Dockerfile:56` (`debian:bookworm-slim`); `docker-compose.yml:142,196` (`JMDN_VERSION:-latest`). Floating/unverified content, reproducibility gap. **Fix:** pin `debian:bookworm-slim@sha256:…`; require an explicit `JMDN_VERSION`.

■■ *INFO*

[INFO] I1 — Self-signed dev CA auto-minted when no certs mounted

Location: `Scripts/docker-entrypoint.sh` cert-gen (CA `-days 3650`, service keys RSA-2048, SAN localhost only). Acceptable for dev; production must mount operator PKI (hook at `docker-compose.yml:287`, commented) and fail closed if certs are missing.

[INFO] I2 — pprof/profiler exposed without auth when enabled

Location: `profiler/profiler.go:7` (`_ "net/http/pprof"`), custom `/debug/fds` runs `sh -c "lsof -p <pid>"` (pid is `os.Getpid()`, **not** user input — no injection). Defaults disabled (`Ports.Profiler: 0`, bind `127.0.0.1`). If enabled, add auth and keep localhost-only.

[INFO] I3 — Positive controls observed

TLS `MinVersion` set to 1.3 (`pkg/gatekeeper/tls.go:77,164`, `seednode/seednode.go:116`); no `InsecureSkipVerify: true` anywhere; SHA-256 throughout (no md5/sha1 for security); Explorer token compare is constant-time (`explorer/api.go:413`, `crypto/subtle`); seed registration rate-limited (5/hr/peer) with registry cap; directMSG uses `io.LimitReader(1MiB)` + read deadline; Thebe read routes validate hex addresses, tx-hash regex, and bound limit/offset (max 500); `DB_OPs` SQL uses parameterized queries (`?` binds; `fmt.Sprintf` only injects constant table names — no user-input SQLi); compose sets `no-new-privileges`, pids/mem/cpu limits, no privileged, no docker.sock mount, healthchecks; `.dockerignore`/`.gitignore` thoroughly exclude `.env`, keys, `jmdn.yaml`, `config/peer.json`, `config/bls.json`.

Recommendations

1. **Fix consensus authentication first (C1, H3).** Bind every vote to `H(chainID‖height‖blockHash‖round‖vote)`, verify signers against a registered committee, deduplicate, and enforce true 2/3 quorum. Until this ships, the chain's core safety property does not hold. Route live consensus through the already-correct `MultiSigManager` + `bft/math.go` helpers.

2. **Close the two remote primitives (C3, H1).** Confine file-transfer paths and cap sizes; remove the JSON tx ingest path and the `V == nil` deployment bypass from public RPC.

3. **Stop authenticating by bind-address.** Enable the existing `gatekeeper` mTLS/token policy for CLI admin, debug, WS, and internal gRPC — `AuthType: none` should not be a default for any write or admin surface. Turn on rate limiting in the default profile.

4. **Purge and rotate committed keys (C2)** and remove weak dev secrets from on-disk config (M1); add pre-commit secret scanning (gitleaks/trufflehog).

5. **Turn CI security scanning back on (M10).** Re-enable `gosec` (diff mode → burn down the 192 backlog → enforce), re-enable Sonar, keep `Scripts/` in scope, and add `govulncheck ./...` + `trivy` as required gates:

```
go install golang.org/x/vuln/cmd/govulncheck@latest && govulncheck ./...
trivy fs .
gitleaks detect --source .
```

Limitations

Static analysis only. This review did **not** execute the node, fuzz inputs, test live exploitability, or observe runtime behavior; it did not assess network-level controls (firewall/LB), the actual deployed configuration on operator hosts, git *history* beyond spot checks, third-party dependency internals, or the block-explorer frontend. Severity ratings assume the affected surface is reachable; several findings are gated behind default `127.0.0.1` binds that a single misconfiguration removes. CVE assessments are heuristic (version age / known history) — confirm with `govulncheck` and `trivy` against the live database. Findings should be validated by the engineering team before remediation is finalized.